CHAPTER I. FUNCTIONAL REQUIREMENTS

The Student Management System provides core features categorized by user roles and permissions (Admin, Student, Lecturer).

1. Authentication & Authorization

Registration: The system does not allow students or lecturers to register their own accounts. Only the Administrator has the right to create accounts based on admission or personnel records. Mandatory fields include: Student ID or Employee ID, National ID (CCCD), Full Name, Date of Birth, Email, Gender, Phone Number, and Faculty/Major.

Username & Password Initialization: The Username will be automatically generated by the system based on the user's Student ID or Employee ID. The initial password is randomly generated and displayed to the Admin on screen for manual delivery to the new user.

Log in: Allows all users to log in using their Username and Password. Upon successful authentication, the system automatically identifies the role (Admin, Student, Lecturer) to display the corresponding interface and features. If the credentials are incorrect, the system will display an error message.

Forgot Password: Users can request a password reset by entering their registered Email or Phone number. Once the system verifies the account exists, the user is allowed to set a new password directly without requiring an OTP or external link.

Log out: Allows users to log out at any time. The system will terminate the session and clear temporary data to ensure security.

Change Password: Users are required to enter their current password before setting a new one. The new password must meet complexity requirements (e.g., minimum 8 characters, including uppercase, numbers, and special characters). The system will require confirmation before saving the changes.

2. User Management

Student Management: The Admin has the right to view, add, update, and delete student records (CRUD). Each student must have a Status. Students can view and update their own personal contact information (Email, Phone Number), but are restricted from modifying identification data such as Student ID, National ID, and affiliated Faculty.

Lecturer Management: Similar to students, the Admin manages all records of the teaching staff. Lecturers can update basic personal information themselves, excluding their affiliated Faculty. The Admin is responsible for assigning lecturers to specific class sections.

3. Academic Management

Faculty Management: The Admin has the right to manage the Faculty catalog, including Faculty Code, Faculty Name, Email, and Phone Number.

Semester/Term Management: The Admin manages academic semesters including start dates, end dates, and event timelines (registration periods, exam weeks). All academic operations are tied to a specific semester.

Course Management: The Admin is allowed to view, create, edit, and delete courses. Course data must include: Course Code, Course Name, Credits, Managing Faculty, and Prerequisite Constraints.

Class/Section Management: A course can be opened as multiple class sections within a specific Semester. The Admin manages classes with: Class Code, Course Code, Semester, Maximum Capacity, Current Enrollment, and a specific Schedule (Day of the week, Start Time to End Time).

View Training Program (Curriculum): Students can view the standard curriculum framework for their major. The system displays the list of courses, credits, and prerequisites, and automatically labels the status as "Completed" or "Incomplete" based on the student's transcript.

4. Grading & Attendance Management

Attendance: The system allows lecturers to take attendance for students in each session, track absences, and automatically calculate the attendance rate.

Gradebook: Lecturers can enter component grades (midterm, assignments, attendance) and final exam grades. The system automatically calculates the final summary grade based on the course's weighted formula, and assigns the letter grade (A, B, C, D, F).

Grade Finalization: Lecturers can save grades as "Draft". Once the grading deadline ends, the Admin or authorized role executes the "Finalize Grades" function. After finalization, grades cannot be altered.

GPA Calculation: The system automatically calculates both Semester GPA and Cumulative GPA based on the finalized grades.

CHAPTER II. NON-FUNCTIONAL REQUIREMENTS

Security: User passwords must be hashed before being stored in the .txt file. The system must terminate the active session and return to the login screen after a defined period of inactivity. The system must strictly comply with Role-Based Access Control (RBAC).

Usability: The system interface must be intuitive and responsive. Error messages must be clear and guide users on how to fix the issue (e.g., highlighting the specific field causing the error or specifying which course causes a schedule conflict).

Reliability: Ensure data integrity and consistency through careful file read/write operations. Data loss is strictly prohibited; all changes must be fully written to the .txt files before the operation is considered complete. The system must handle unexpected errors gracefully to prevent file corruption.

Performance: As the system is built entirely with Python and relies on .txt files for data storage, concurrent write operations are not supported. The system is intended for small-scale use within a single institution. The system must respond to any user action within 3 seconds under normal operation. File read/write operations must complete without data corruption, and the system must handle file access errors gracefully with appropriate error messages.

Maintainability: The system needs to be developed using a modular architecture to easily facilitate future upgrades and feature expansions. Code must be documented and follow standard naming conventions.

Design Constraints: The Student Management System is built entirely with Python, relying on .txt files for reading, writing, and managing all student data.

CHAPTER III. DATA FLOW DIAGRAM (DFD)

1. Overview of DFD

This document provides a detailed description of two levels of the Data Flow Diagram (DFD) for the Student Management System (SMS). A DFD is a modeling tool used to represent information flows, processing functions, external entities, and data stores within a system.

The two DFD levels included in this document are

- DFD Level 0 (Context Diagram): A high-level overview treating the entire system as a single process, showing its interactions with external entities.
- DFD Level 1: Decomposes the main process into 4 sub-processes, showing more detailed data flows and the system's data stores.

2. DFD Diagram Level 0 – Context Diagram

2.1 Overview

DFD Level 0, also known as the Context Diagram, represents the entire Student Management System as a single process labeled "0 – Student Management System (SMS)". This diagram defines the system boundary and describes the interactions between the system and three external entities: Admin, Student, and Lecturer.

2.2 External Entities

Entity	Role	Description
Admin	System Administrator	Manages the entire system with the highest level of access, responsible for configuration, user management, and academic data setup.
Student	End User	Uses the system to view academic transcripts, check grades, and manage personal information.
Lecturer	End User	Uses the system to record attendance, enter grades for assigned classes, and view enrolled student lists.

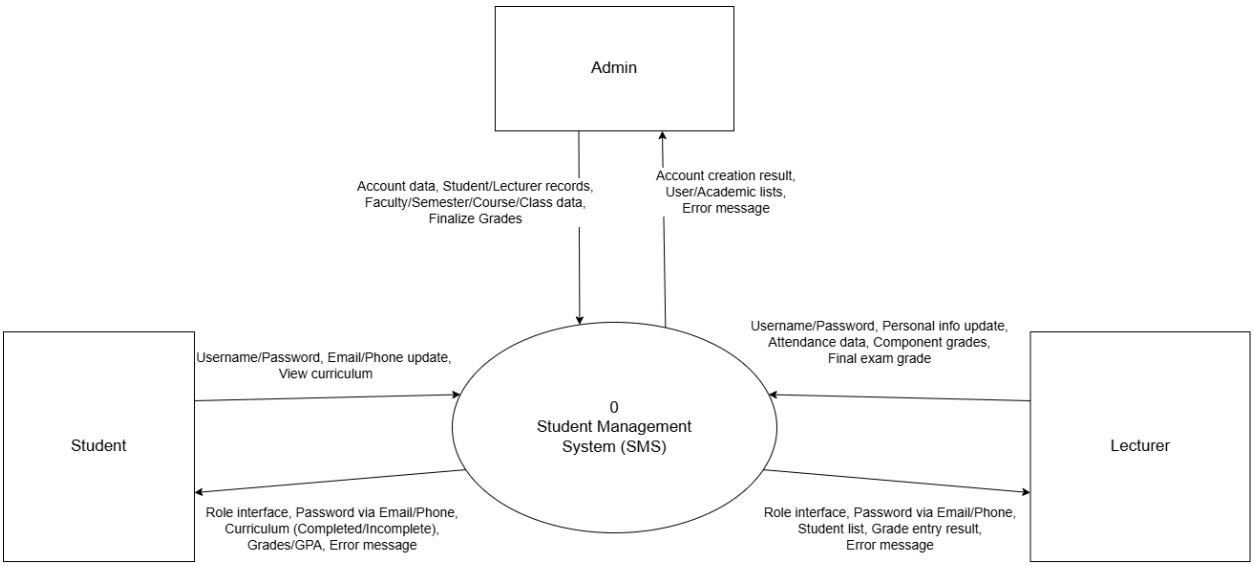

Image 1: DFD diagram level 0

3. DFD Diagram Level 1 – Main Process Decomposition

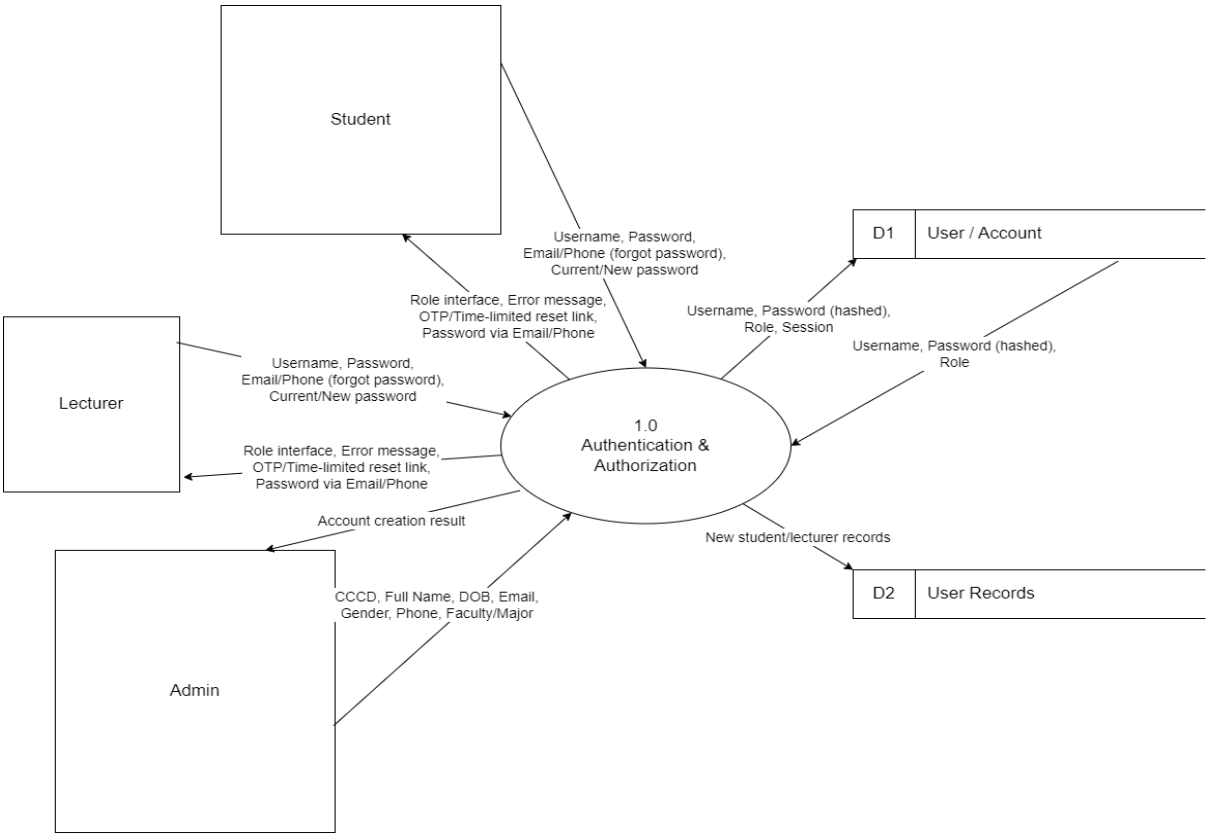

Image 2: Process 1

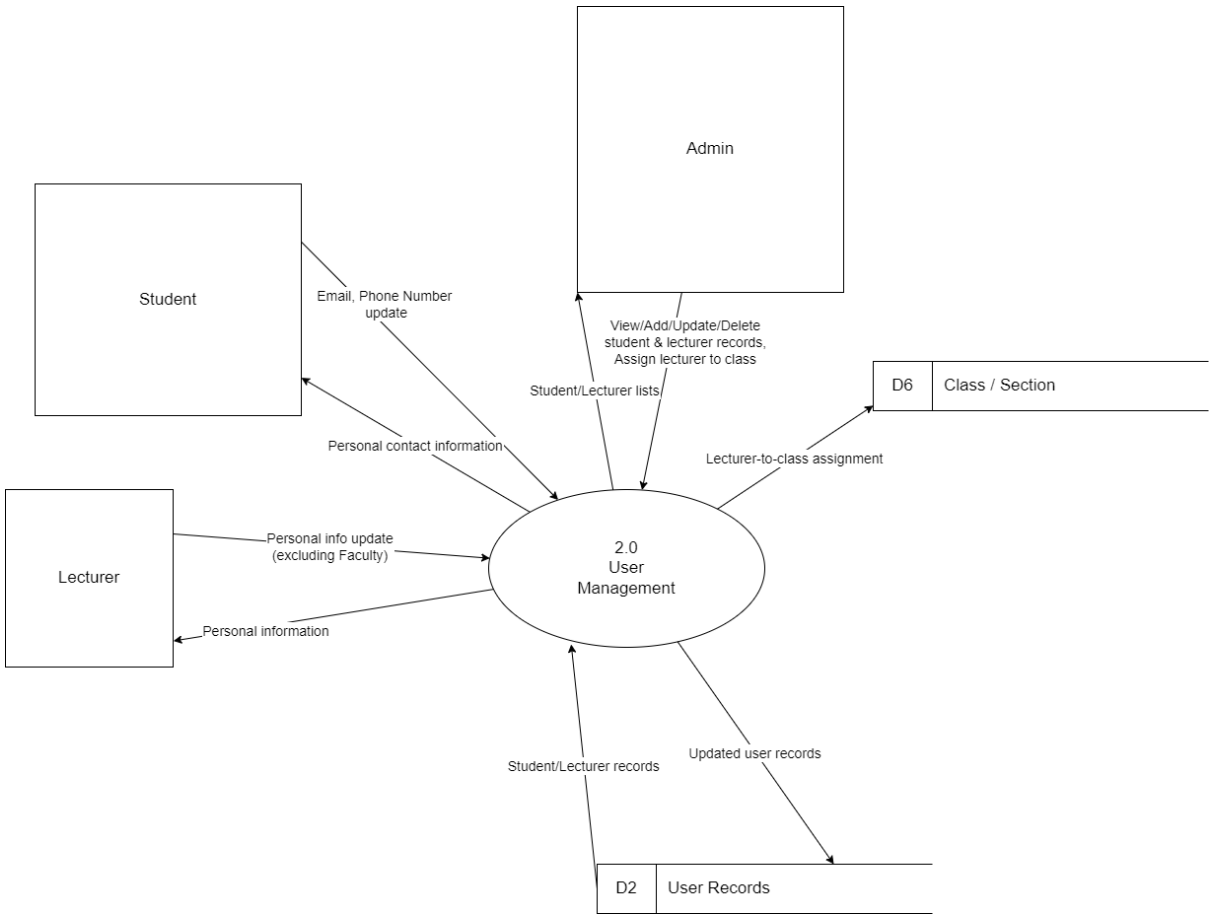

Image 3: Process 2

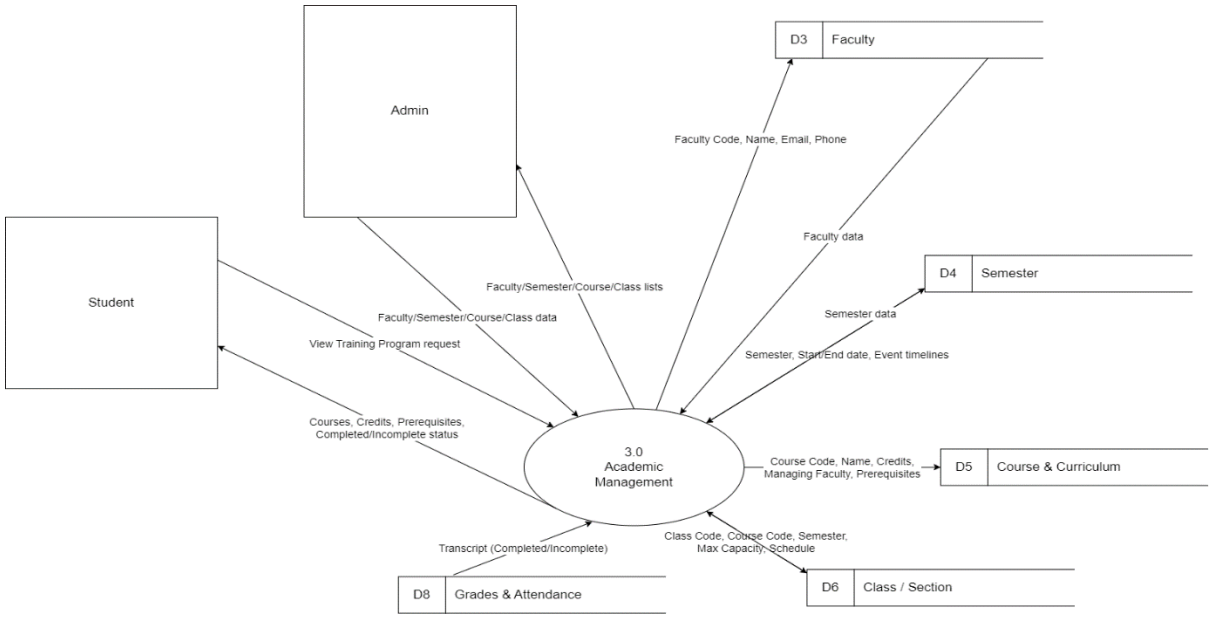

Image 4: Process 3

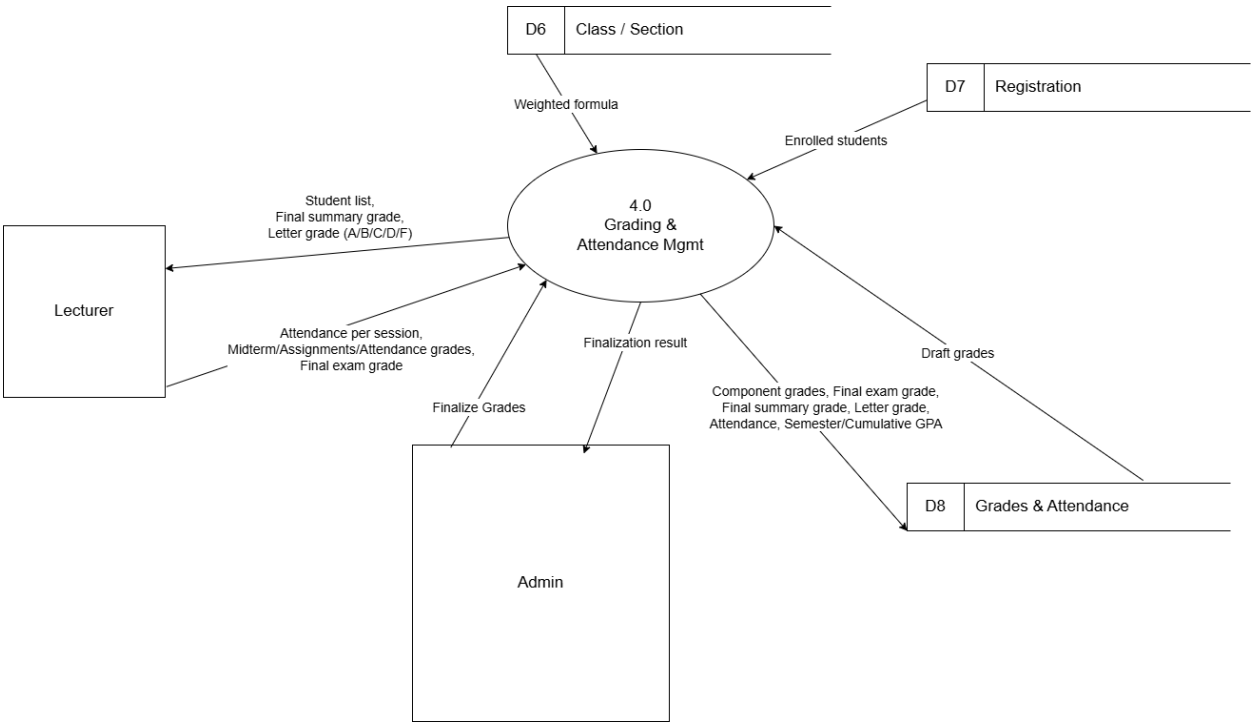

Image 5: Process 4

3.1 Overview

DFD Level 1 decomposes the central SMS process into 4 sub-processes, showing detailed data flows between external entities (Admin, Student, Lecturer), processing functions, and 8 data stores.

3.2 Processes

Process ID	Process Name	Functional Description
1.0	Authentication & Authorization	Handles identity verification and role-based access control for all users (Admin, Student, Lecturer). Manages login sessions, password reset.

2.0	User Management	Manages user profiles: creating, updating, and deleting student and lecturer records. Handles personal contact information updates.
3.0	Academic Management	Manages all academic data structures including faculties, semesters, courses, curricula, and class sections. Only Admin can create or modify academic data.
4.0	Grading & Attendance Mgmt	Manages per-session attendance records, component grade entry, final exam grades, and GPA calculation (semester and cumulative).

Table 2

3.3. Data Stores

Store ID	Name	Primary Contents
D1	User / Account	Username, hashed password, user role, session information.
D2	User Records	Detailed profiles: full name, national ID, date of birth, gender, email, phone number, faculty/major assignment.
D3	Faculty	Faculty code, faculty name, contact email, phone number.
D4	Semester	Semester code, start/end dates, event milestone timelines, registration portal status.

D5	Course & Curriculum	Course code, course name, credit hours, managing faculty, prerequisite courses.
D6	Class / Section	Class code, linked course code, semester, maximum capacity, current enrollment count, class schedule (day/time).
D7	Registration	Student-class enrollment records: student ID, class ID, registration status.
D8	Grades & Attendance	Component grades, exam grades, final summary grade, letter grade (A/B/C/D/F), attendance records, semester GPA, cumulative GPA.

Table 3

CHAPTER IV. USE CASE DIAGRAM

1. Use Case Diagram

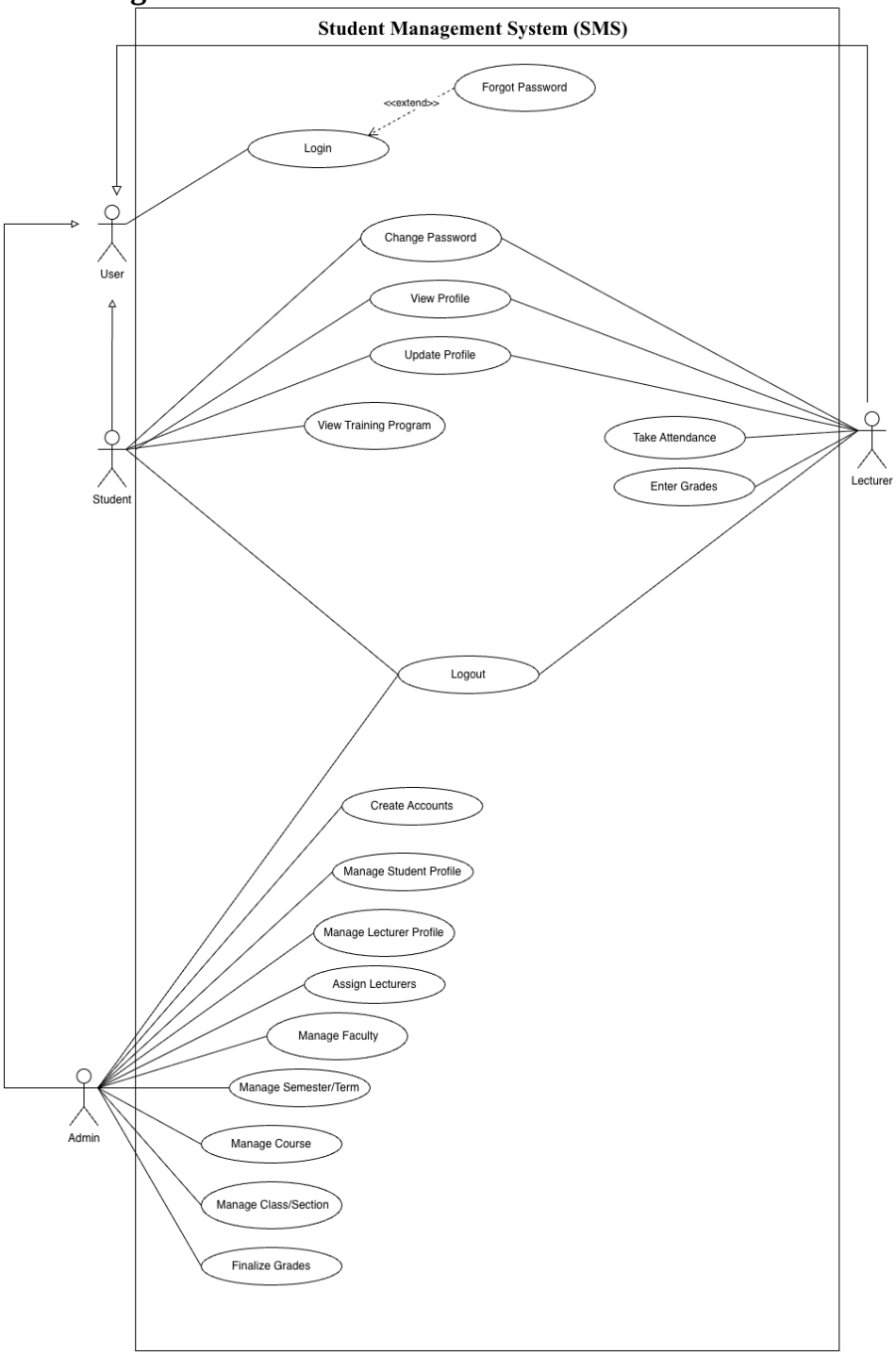

Image 6: Use case diagram

2. List Of Actor

No.	Actor	Description
1.	User	Represents a user who is not logged in and can perform authentication (log in, forgot password).
2.	Student	Student can change password, view and update personal profile, view the training program, and log out.
3.	Lecturer	Lecturer can change password, view and update personal profile, take attendance, enter grades, and log out.
4.	Admin	Admin can create accounts, manage user profiles, assign lecturers, manage academic data (faculty, semester, course, class), finalize grades, and log out.

Table 5 : Actor

3. List Of Use Case

No.	Use case	Description
1.	Create Accounts	Only the admin has the right to create accounts based on admission or personnel records.
2.	Log in	Allows users to log in, the system verifies credentials and redirects based on role.
3.	Forgot Password	Users request a password reset via their registered Email or Phone number to set a new password directly.
4.	Log out	Allows users to log out of the system at any time.
5.	Change Password	Users enter the current password, set a valid new complex password, and confirm.

6.	View Profile	Allows users to view their personal information.
7.	Update Profile	Allows users to update their editable personal contact information.
8.	Manage Student Profile	Allows the admin to view, add, update, and delete student records.
9.	Manage Lecturer Profile	Allows the admin to view, add, update, and delete lecturer records.
10.	Assign Lecturers	Allows the admin to assign lecturers to specific class sections.
11.	Manage Faculty	Allows the admin to manage the Faculty catalog.
12.	Manage Semester/Term	Allows the admin to manage academic semesters and event timelines.
13.	Manage Course	Allows the admin to view, create, edit, and delete courses.
14.	Manage Class/Section	Allows the admin to manage class sections within a specific semester.
15.	View Training Program	Allows students to view the standard curriculum framework for their major
16.	Take attendance	Allows lecturers to take attendance for students in each session.
17.	Enter Grades	Allows lecturers to enter component and final exam grades, and save them as drafts.

18.	Finalize Grades	Allows the admin to finalize course grades, after which they cannot be altered.

Table 6 : Use case

4. Create Accounts

4.1 Summary

Only the Admin has the right to create accounts based on admission or personnel records. Students and lecturers cannot self-register.

4.2 Flow of Events

4.2.1. Main Flow

1. The admin inputs mandatory fields: Student ID or Employee ID, National ID (CCCD), Full Name, Date of Birth, Email, Gender, Phone Number, and Faculty/Major.
2. The system automatically generates the Username based on the user's Student ID or Employee ID.
3. The system randomly generates an initial password.
4. The system displays the generated password on screen for the Admin to manually deliver to the new user.
5. The system hashes the password and saves the new account record into the database (.txt file).

4.2.2 Alternative Flow

If input is invalid or a duplicate ID exists, the system shows an error and notifies the Admin.

4.3 Special Requirements

All user data must be managed using Python and **.txt** files. Passwords must be hashed before storage.

4.4 Preconditions

The admin is logged into the system.

4.5 Postconditions

A new account is created with a system-generated username and password.

4.6 Extension Points

None.

5. Log in

5.1 Summary

Allows all users to log in to the system using their Username and Password.

5.2 Flow of Events

5.2.1 Main Flow

1. The user enters their Username and Password.
2. The system validates the credentials against the .txt database.
3. The system identifies the user role (Admin, Student, Lecturer).
4. The system redirects the user to the appropriate role-based interface.

5.2.2 Alternative Flow

If the credentials are incorrect, the system displays an error message.

5.3 Special Requirements

The system must strictly comply with Role-Based Access Control (RBAC).

5.4 Preconditions

The user has a valid registered account.

5.5 Postconditions

The user is logged into the system and a session is initiated.

5.6 Extension Points

Forgot Password

6. Forgot Password

6.1 Summary

Allows users to request a password reset by entering their registered Email or Phone number when they cannot log in.

6.2 Flow of Events

6.2.1 Main Flow

1. The user selects the "Forgot Password" option.
2. The system prompts the user to enter their registered Email or Phone number.
3. The system verifies if the provided Email or Phone number exists in the .txt database.
4. Once verified, the system allows the user to set a new password directly on the screen.
5. The system validates the password complexity, hashes the new password, and updates the user’s record in the .txt file.
6. The system displays a success message.

6.2.2 Alternative Flow
If the entered Email or Phone number is not found in the system, the system displays an error message.

6.3 Special Requirements
This process does not require an OTP or external link as per the updated system constraints.

6.4 Preconditions
The user is not logged in and has a valid Email or Phone number registered in the system.

6.5 Postconditions
The user's password is changed in the **.txt** database.

6.6 Extension Points
None

7. Log out
7.1 Summary
Allows users to log out of the system at any time.

7.2 Flow of Events
7.2.1 Main Flow

1. The user selects the logout option.
2. The system terminates the active session.
3. The system clears temporary data to ensure security.
4. The system redirects the user to the login page.

7.2.2 Alternative Flow
None.

7.3 Special Requirements
The system must automatically return to the login screen after a defined period of inactivity.

7.4 Preconditions
The user is currently logged in.

7.5 Postconditions
The user is logged out safely.

7.6 Extension Points
None.

8. Change Password

8.1 Summary

Allows users to change their current password.

8.2 Flow of Events

8.2.1 Main Flow

1. The user enters their current password.
2. The user enters a new password and re-enters it to confirm.
3. The system validates the current password.
4. The system checks if the new password meets complexity rules.
5. The system confirms the new password entries match before saving.
6. The system hashes and updates the password in the .txt file.

8.2.2 Alternative Flow

If the current password is incorrect or the new password fails complexity checks, the system displays an error message.

8.3 Special Requirements

The new password must meet complexity requirements (e.g., minimum 8 characters, including uppercase, numbers, and special characters).

8.4. Preconditions

The user is logged in.

8.5. Postconditions

Password is updated.

8.6 Extension Points

None.

9. View Profile

9.1 Summary

Allows users to view their personal information.

9.2 Flow of Events

9.2.1 Main Flow

1. The user navigates to the profile section.
2. The system retrieves the user's data from the .txt file.
3. The system displays the profile details on the interface.

9.2.2 Alternative Flow

None.

9.3 Special Requirements
None.

9.4 Preconditions
The user is logged in.

9.5 Postconditions
Profile is displayed.

9.6 Extension Points
None.

10. Update Profile

10.1 Summary
Allows students and lecturers to update their personal contact information.

10.2 Flow of Events

10.2.1 Main Flow

1. The user modifies their editable fields (Email, Phone Number).
2. The system validates the input format.
3. The system saves the changes to the .txt database.

10.2.2 Alternative Flow
If the input format is invalid, the system displays an error message.

10.3 Special Requirements
Students are restricted from modifying Student ID, National ID, and Faculty. Lecturers cannot modify their affiliated Faculty.

10.4 Preconditions
The user is logged in.

10.5 Postconditions
The user's contact information is updated in the **.txt** file.

10.6 Extension Points
None.

11. Manage Student Profile

11.1 Summary
Allows the Admin to view, add, update, and delete student records (CRUD).

11.2 Flow of Events

11.2.1 Main Flow

1. The admin accesses the student management module.
2. The admin selects a CRUD action (view, add, update, delete).
3. The admin inputs or edits the student data, ensuring a "Status" is assigned.
4. The system updates the corresponding .txt file.

11.2.2 Alternative Flow

If input data is missing/invalid, or a duplicate ID exists, the system displays an error.

11.3 Special Requirements

Each student must have a valid Status.

11.4 Preconditions

The admin is logged into the system.

11.5 Postconditions

Student records are successfully modified in the **.txt** file.

11.6 Extension Points

None.

12. Manage Lecturer Profile

12.1 Summary

Allows the Admin to view, add, update, and delete teaching staff records.

12.2 Flow of Events

12.2.1 Main Flow

1. The admin accesses the lecturer management module.
2. The admin performs CRUD operations on lecturer data.
3. The system updates the lecturer database in the .txt file.

12.2.2 Alternative Flow

If input data is invalid, the system prevents the update and shows an error.

12.3 Special Requirements

None.

12.4 Preconditions

The admin is logged into the system.

12.5 Postconditions

Lecturer records are successfully updated in the .txt file.

12.6 Extension Points

None.

13. Assign Lecturers

13.1 Summary

Allows the admin to assign lecturers to specific class sections.

13.2 Flow of Events

13.2.1 Main Flow

1. The admin selects a specific class section.
2. The admin assigns a valid lecturer to that section.
3. The system saves the assignment into the .txt file.

13.2.2 Alternative Flow

If the lecturer is unavailable or the class does not exist, the system shows an error.

13.3 Special Requirements

None.

13.4 Preconditions

The admin is logged into the system, and class sections/lecturers must exist.

13.5 Postconditions

The lecturer is successfully assigned to the class section.

13.6 Extension Points

None.

14. Manage Faculty

14.1 Summary

Allows the Admin to manage the Faculty catalog (Code, Name, Email, Phone).

14.2 Flow of Events

14.2.1 Main Flow

1. The admin accesses the faculty catalog.
2. The admin views, adds, updates, or deletes faculty information.
3. The system writes the updates to the .txt file.

14.2.2 Alternative Flow

If the faculty code already exists during creation, the system shows an error.

14.3 Special Requirements
None.

14.4 Preconditions
The admin is logged into the system.

14.5 Postconditions
Faculty catalog data is updated.

14.6 Extension Points
None.

15. Manage Semester/Term

15.1 Summary
Allows the Admin to manage academic semesters and event timelines.

15.2 Flow of Events

15.2.1 Main Flow

1. The admin accesses the semester management module.
2. The admin sets or updates the start date, end date, and event timelines (e.g., exam weeks).
3. The system saves the semester details to the .txt file.

15.2.2 Alternative Flow
If dates overlap illogically, the system shows an error.

15.3 Special Requirements
All academic operations must be tied to a specific semester.

15.4 Preconditions
The admin is logged into the system.

15.5 Postconditions
The semester timeline is established.

15.6 Extension Points
None.

16. Manage Course

16.1 Summary
Allows the admin to view, add, update, and delete courses.

16.2 Flow of Events

16.2.1 Main Flow

1. The admin accesses course management.
2. The admin inputs course details: Code, Name, Credits, Managing Faculty, and Prerequisites.
3. The system validates and saves the data to the .txt file.

16.2.2 Alternative Flow

If mandatory fields are missing, the system blocks the action.

16.3 Special Requirements

Prerequisite constraints must be accurately recorded.

16.4 Preconditions

The admin is logged in. Faculties must exist.

16.5 Postconditions

The course catalog is updated.

16.6 Extension Points

None.

17. Manage Class/Section

17.1 Summary

Allows the Admin to manage class sections opened within a specific semester.

17.2 Flow of Events

17.2.1 Main Flow

1. The admin selects a course and semester.
2. The admin defines the Class Code, Maximum Capacity, and Schedule (Day/Time).
3. The system saves the class section data to the .txt file.

17.2.2 Alternative Flow

If the class code is duplicated within the semester, the system shows an error.

17.3 Special Requirements

A course can be opened as multiple class sections.

17.4 Preconditions

The admin is logged in. Courses and semesters must exist.

17.5 Postconditions
The class section is configured in the system.

17.6 Extension Points
None.

18. View Training Program

18.1 Summary
Allows students to view the standard curriculum framework for their major.

18.2 Flow of Events

18.2.1 Main Flow

1. The student navigates to the Training Program section.
2. The system reads the curriculum .txt file and the student's transcript .txt file.
3. The system displays the courses and automatically labels the status as "Completed" or "Incomplete" based on the transcript.

18.2.2 Alternative Flow
None.

18.3 Special Requirements
Automated labeling of completion status.

18.4 Preconditions
The student is logged in.

18.5 Postconditions
The annotated training program is displayed.

18.6 Extension Points
None.

19. Take Attendance

19.1 Summary
Allows lecturers to record student attendance in each session.

19.2 Flow of Events

19.2.1 Main Flow

1. The lecturer selects a specific class session.
2. The lecturer tracks and marks student absences.
3. The system automatically calculates the attendance rate.
4. The system saves the attendance data to the .txt file.

19.2.2 Alternative Flow
If the class session is invalid, the system displays an error.

19.3 Special Requirements
Automatic calculation of the attendance rate.

19.4 Preconditions
The lecturer is logged in and assigned to the class.

19.5 Postconditions
Attendance records are successfully updated.

19.6 Extension Points
None.

20. Enter Grades

20.1 Summary
Allows lecturers to enter component and final exam grades.

20.2 Flow of Events

20.2.1 Main Flow

1. The lecturer selects a class and enters component grades (midterm, assignments, attendance) and the final exam grade.
2. The system automatically calculates the final summary grade based on the weighted formula and assigns a letter grade (A, B, C, D, F).
3. The lecturer saves the grades as "Draft".
4. The system writes the draft grades to the .txt file.

20.2.2 Alternative Flow
If the grades have already been finalized by the Admin, the system blocks the lecturer from editing.

20.3 Special Requirements
Automatic calculation of summary and letter grades.

20.4 Preconditions
The lecturer is logged in and assigned to the class.

20.5 Postconditions
Draft grades are saved.

20.6 Extension Points
None.

21. Finalize Grades

21.1 Summary

Allows the Admin to finalize course grades once the grading deadline ends.

21.2 Flow of Events

21.2.1 Main Flow

1. The admin verifies the grading deadline has passed.
2. The admin executes the "Finalize Grades" function for the draft grades.
3. The system locks the grades.
4. The system automatically calculates the Semester GPA and Cumulative GPA.
5. The system saves the finalized grades and GPAs to the .txt file.

21.2.2 Alternative Flow

If the deadline has not passed, the system prevents finalization.

21.3 Special Requirements

After finalization, grades cannot be altered under any circumstances.

21.4 Preconditions

The admin is logged in. Lecturers have submitted draft grades.

21.5 Postconditions

Grades are locked permanently and GPAs are calculated.

21.6 Extension Points

None

CHAPTER V. CLASS DIAGRAM

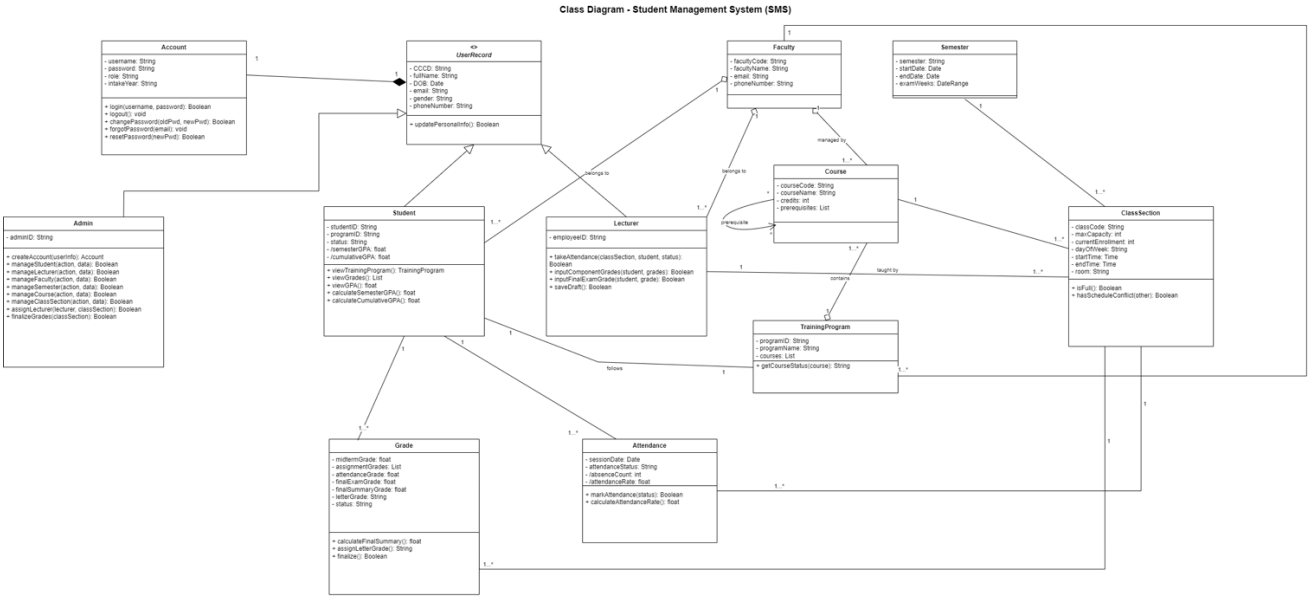

Image 7: Class diagram

CHAPTER VI. DATA MODEL

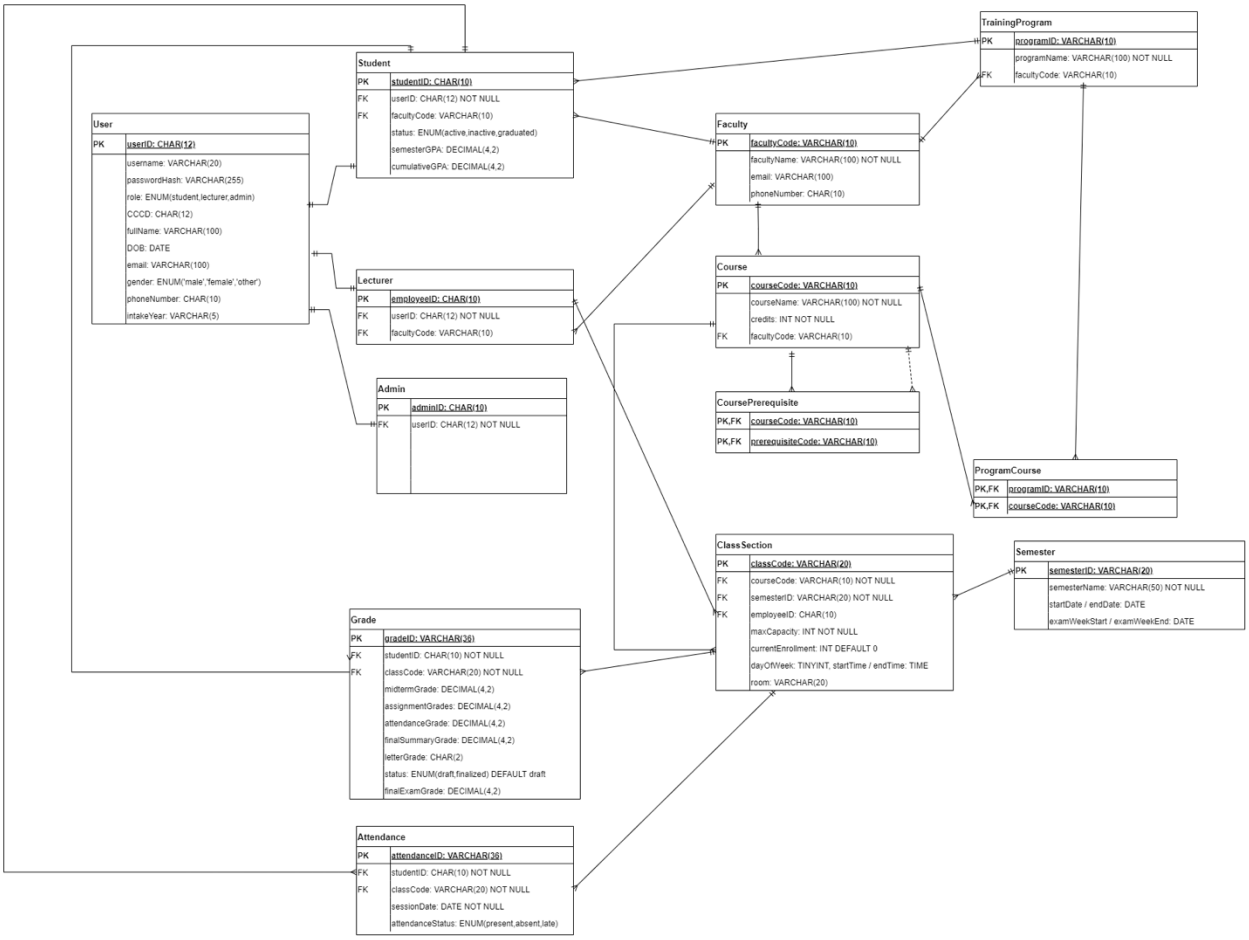

Image 8: Data model

CHAPTER VII. INTERFACE DESIGN DESCRIPTION

1. Overall Interface Overview

The Student Management System (SMS) is implemented as a modern application with a responsive Graphical User Interface (GUI), developed entirely using Python. Despite relying on .txt files for reading, writing, and managing all data, the system strictly avoids standard text-based console inputs. Instead, users interact with the system through visual forms, dashboards, and navigation menus. Based on user roles (Admin, Student, Lecturer), the system complies with Role-Based Access Control (RBAC) to provide separate menus with appropriate functions.

2. Main Menu & Authentication Design

When the application starts, the system displays a main authentication menu that serves as the entry point for all users. The main menu provides the following options:

- **Log in** using a Username and Password.
- **Forgot Password** to request a password reset via registered Email or Phone number. Once the system verifies the account exists in the .txt database, the user is allowed to set a new password directly on the screen without requiring an OTP or external link.
- Upon successful authentication, the system automatically identifies the role (Admin, Student, Lecturer) to display the corresponding interface and features.

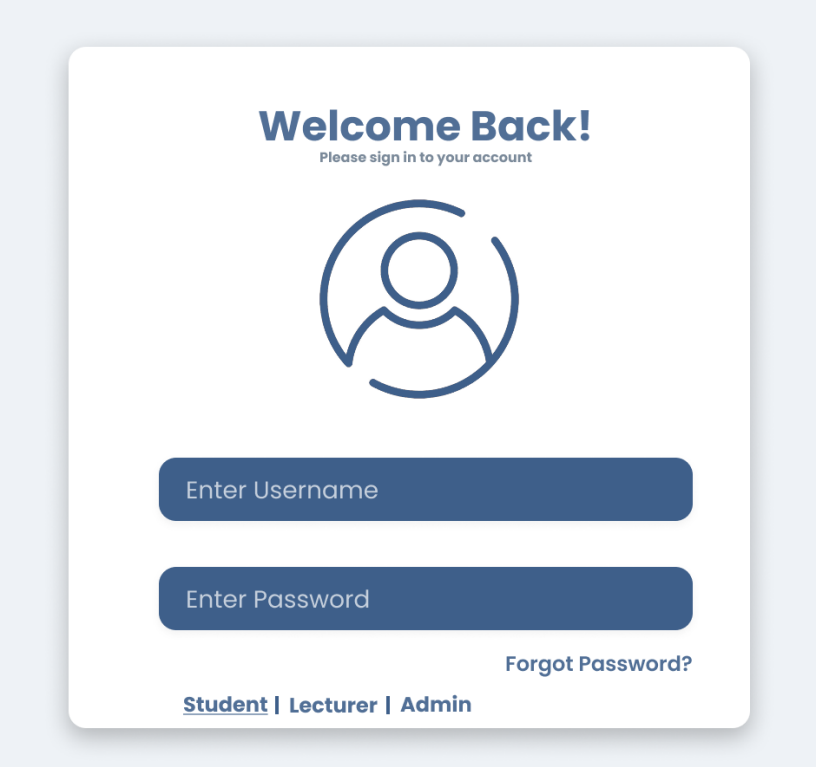

Image 9: Student Authentication Interface

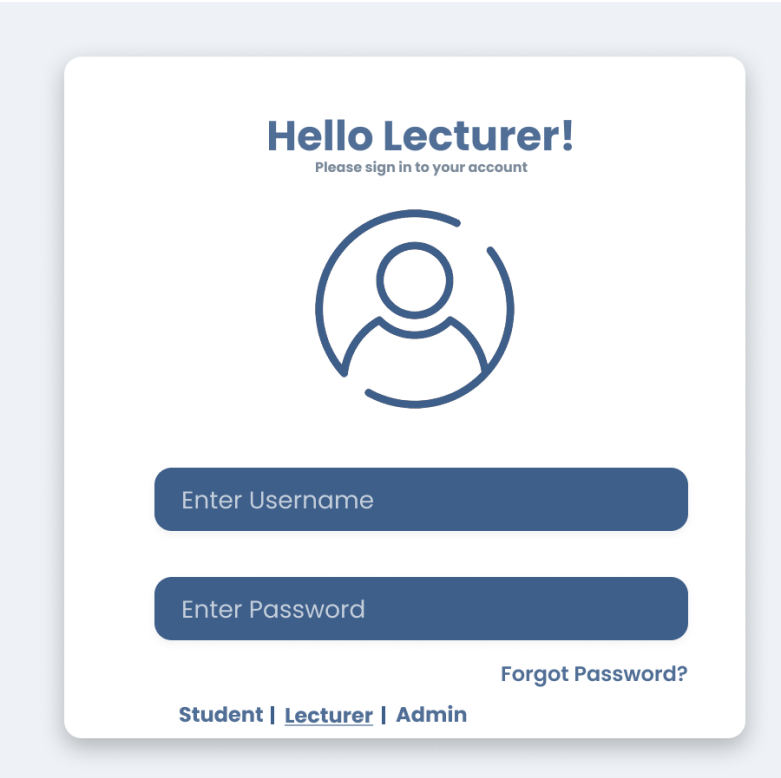

Image 10: Lecturer Authentication Interface

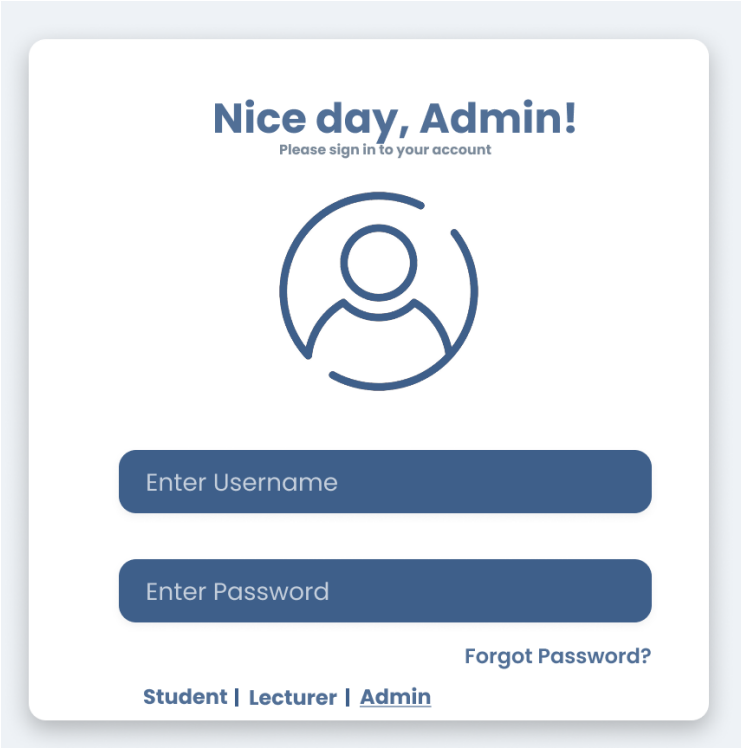

Image 11: Administrator Authentication Interface

3. Role-Based Menu Design

To ensure clear separation of responsibilities and secure access control, the system implements role-based menus. Each role is provided with a distinct set of functions.

3.1 Administrator Interface

The administrator interface allows administrators to manage academic data, system configurations, and user accounts. Main functions provided to administrators include:

- Creating accounts based on admission or personnel records.
- Managing student and lecturer profiles (view, add, update, delete).
- Assigning lecturers to specific class sections.
- Managing the Faculty catalog, Semesters/Terms, Courses, and Class/Sections.
- Executing the "Finalize Grades" function once the grading deadline ends.
- Changing passwords and logging out of the system.

Image 12: Administrator Dashboard

3.2 Student Interface

The student interface is designed to support academic tracking. Students can perform the following actions:

- Viewing and updating their own personal contact information (Email, Phone Number).
- Viewing the standard curriculum framework for their major, with statuses automatically labeled as "Completed" or "Incomplete" based on their transcript data.
- Changing passwords and logging out of the system.

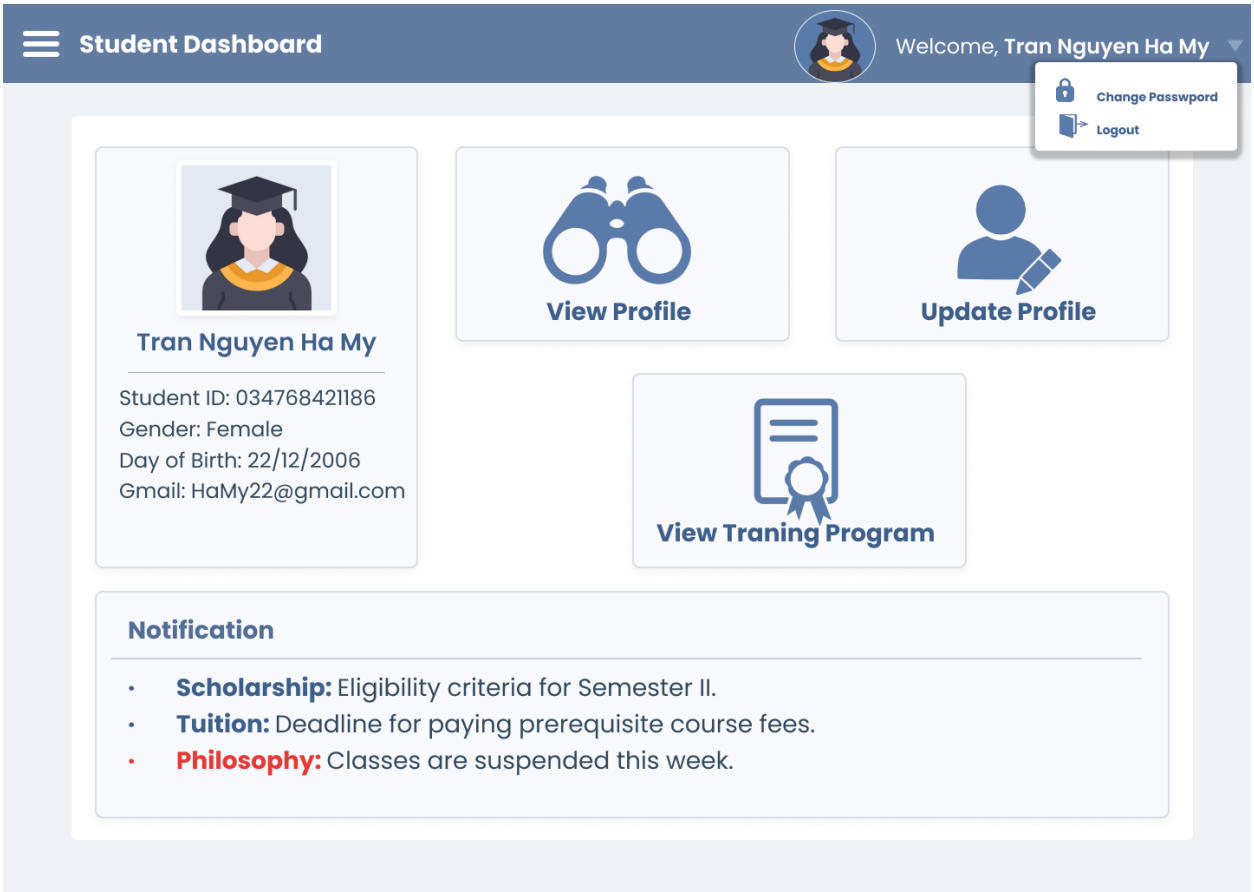

Image 13: Student Dashboard

3.3 Lecturer Interface

The lecturer interface provides basic teaching-related functionalities. Lecturers are able to:

- Updating basic personal information, excluding their affiliated Faculty.
- Taking attendance for students in each session and tracking absences.
- Entering component grades and final exam grades, and saving grades as "Draft".
- Changing passwords and logging out of the system.

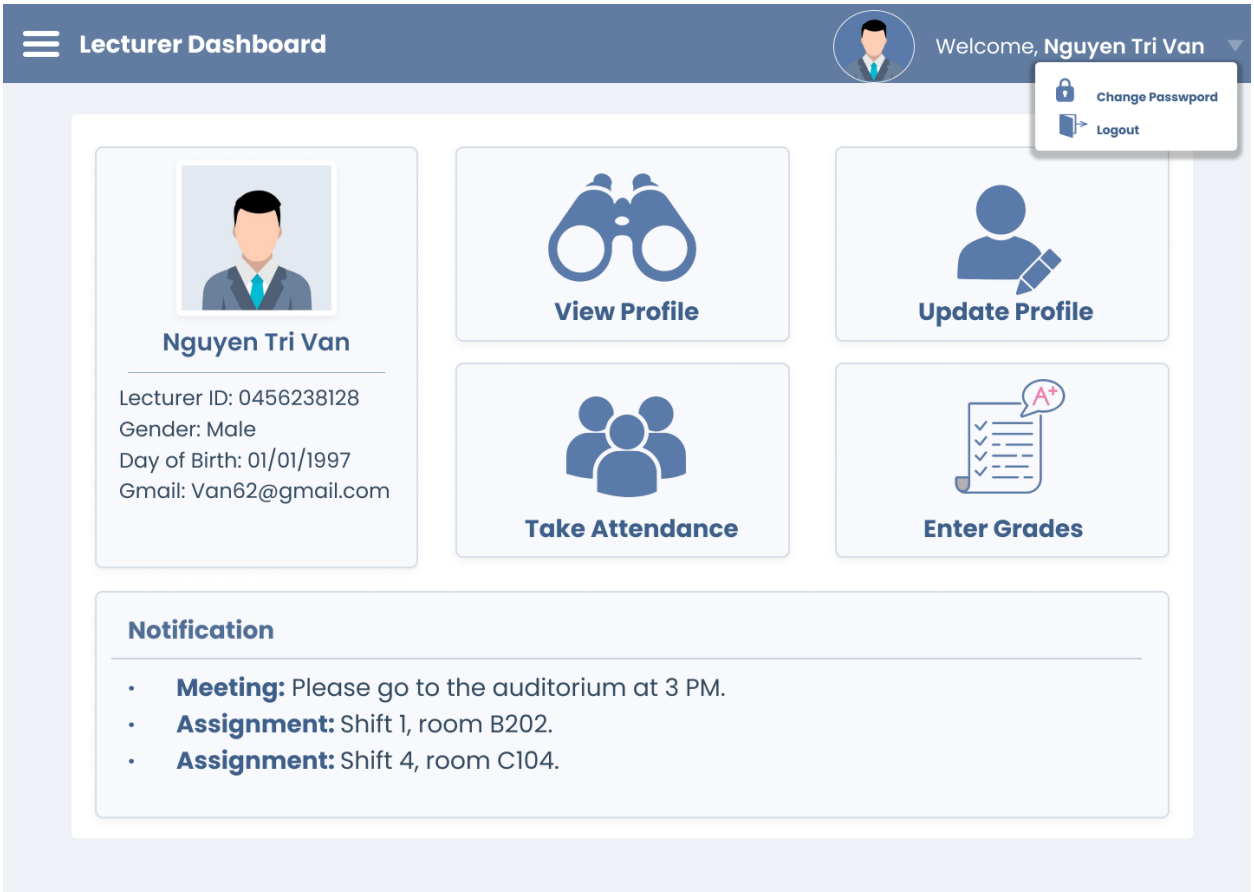

Image 14: Lecturer Dashboard

4. **User Interaction and System Feedback**

The system provides immediate and clear feedback after each user action.

- If an operation is completed successfully, the system proceeds with the action, such as requiring confirmation before saving password changes or displaying a success message.
- If an operation fails or credentials are incorrect, the system will display an error message.
- Error messages must be clear and guide users on how to fix the issue. The system provides specific visual feedback, such as highlighting the specific field causing the error (e.g., highlighting the duplicate ID field when adding a new user, or invalid grade input).

5. **Interface Design Rationale**

A responsive Graphical User Interface (GUI) built with Python was selected for this system due to the following reasons:

- The non-functional requirements explicitly mandate that the system interface must be intuitive and responsive.
- The system requires advanced visual error handling, such as highlighting specific fields causing errors, which is effectively implemented in GUI environments rather than standard console applications.
- The visual interface effectively manages the display of complex academic data, such as curriculum frameworks and gradebooks, seamlessly integrating with the backend .txt file data management system.